Operating Systems

ASSIGNMENT 1

CO1: *Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.*

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Operating Systems for a Smart Hospital

A 500-bed hospital is replacing its computer systems. It needs: bedside patient monitors that must raise an alarm within 10 milliseconds of an abnormal reading; nurses' handheld tablets; doctors' desktop workstations; a central database server for patient records that must never stop; an imaging server that processes MRI scans using many processors; and a link to two partner hospitals that share records.

The IT team has proposed using one general-purpose operating system everywhere. The clinical staff are concerned that alarms may be delayed, the database may stop during updates, and the tablets' batteries will not last a shift.

The number of devices may grow:

1 ward (40 monitors, 10 tablets) → 1 hospital (500 monitors, 120 tablets, 80 workstations) → a group of 3 hospitals sharing records

You are required to evaluate the requirements of every part of the system, judge which type and structure of operating system suits each, examine how programs obtain OS services through system calls, and defend a complete operating-system plan for the hospital.

Design Challenge:

The hospital is adding remote patient monitoring at home and a cloud-based records service.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate the Requirements of Each Computing Environment 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate what each part of the hospital needs from its operating system.

Discuss:

- Response-time, reliability, throughput, battery and security requirements of each device
- Which requirements conflict (for example: fast response and many users on one machine)
- Which requirements are critical to patient safety
- How the lessons from the history of operating systems (batch, time-sharing, personal, mobile) apply to each environment

You are expected to **rank the requirements of each environment and justify the ranking**.

Alternative Concept and Complexity Analysis

If the whole hospital used a single general-purpose operating system on every device, instead of a different type of OS for each environment, what would the solution be? Evaluate the worst-case alarm response time (unbounded scheduling delay versus a guaranteed deadline), the memory needed on each device, and the administration effort.

Evidence Expected

A ranked requirement table for every environment with a justification for each ranking.

TASK 2 – Judge the Type of Operating System for Each Environment **3 Marks**

Bloom's Level: K5 – Evaluate

Compare the types of operating systems for the hospital.

Discuss:

- Mainframe, desktop, multiprocessor, distributed, clustered, real-time (hard and soft) and handheld systems
- Which type best fits the monitors, tablets, workstations, database server, imaging server and partner link
- Symmetric versus asymmetric multiprocessing for the imaging server
- A clustered system with fail-over for the database server

Alternative Concept and Complexity Analysis

If the database server used a single large multiprocessor machine instead of a cluster of smaller machines with fail-over, what would the solution be? Evaluate the availability (a single point of failure versus fail-over within seconds), the cost, and how throughput grows with the number of processors (time) and the memory available (space).

Evidence Expected

A justified choice of operating-system type for each environment, with the rejected alternatives and reasons.

TASK 3 – Evaluate Operating-System Structures **4 Marks**

Bloom's Level: K5 – Evaluate

Evaluate the possible structures of the operating system.

Discuss:

- Simple (monolithic), layered, microkernel, modular (loadable kernel modules) and hybrid structures
- The effect of each structure on reliability, performance and ease of updating
- Which structure suits the patient monitors and which suits the database server
- How Linux, Windows, Android and a real-time OS such as FreeRTOS or QNX are structured

Alternative Concept and Complexity Analysis

If the bedside monitors used a monolithic kernel instead of a microkernel, what would the solution be? Evaluate the time cost of a service request (one system call versus message passing between servers), the memory taken by the kernel, and the effect of a faulty driver on the whole system.

Evidence Expected

A comparison of structures against the hospital's requirements, with a defended choice for two environments.

TASK 4 – Evaluate the Services an Operating System Must Provide **4 Marks**

Bloom's Level: K5 – Evaluate

Evaluate the services each environment needs.

Discuss:

- User interface, program execution, I/O operations, file systems, communication, error detection, resource allocation, accounting, protection and security
- Which services can be removed from the bedside monitor to make it smaller and faster
- Which services are essential for the database server
- How the OS protects one patient's records from another user's programs

Alternative Concept and Complexity Analysis

If the tablets used a full desktop operating system with every service enabled instead of a reduced handheld OS, what would the solution be? Evaluate the memory and storage used by the extra services (space), the start-up time and the battery life (time).

Evidence Expected

A service table for each environment with justified inclusions and exclusions.

TASK 5 – Examine System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Examine how a program obtains OS services.

Discuss:

- The categories of system calls (process control, file management, device management, information maintenance, communication, protection)
- What happens step by step when a program calls read(): library call, trap, mode switch, kernel work, return
- How parameters are passed to the kernel (registers, a block in memory, the stack)
- How strace shows the system calls made by a program

Alternative Concept and Complexity Analysis

If programs called the C library functions fopen, fread and fwrite instead of the open, read and write system calls directly, what would the solution be? Evaluate the number of system calls for a file of n bytes (about n / 4096 with library buffering versus n / b for a buffer of b bytes) and the extra buffer space used by the library.

Evidence Expected

A step-by-step explanation of a system call with strace evidence from a real program.

TASK 6 – Evaluate the Cost of System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the number of system calls affects performance.

Discuss:

- Why each system call costs much more than an ordinary function call
- How the buffer size changes the number of read() and write() calls when the database copies a file

- How library buffering (fread, fwrite) reduces the number of system calls
- The best buffer size for the hospital's backup program, using your measurements

Complete the Code

Complete TODO 1 below using the read() and write() system calls, and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
   bufsize bytes. Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;
    // TODO 1: Loop: read up to bufsize bytes into buf (count every read call in calls);
    //   stop when read returns 0 (end of file) or -1; write exactly the n bytes that
    //   were read

    close(in);
    close(out);
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");     /* a 10,000-byte test file */
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
    fclose(f);

    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

Expected output:

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Run the program under strace -c and compare the number of read() calls reported with the program's output.

Alternative Concept and Complexity Analysis

If the backup program used memory-mapped files (mmap) instead of read() and write() with a buffer, what would the solution be? Evaluate the number of system calls (a few versus about n / b), the copying between kernel and user space, and the memory used for very large files.

Evidence Expected

Measured system-call counts and times for different buffer sizes, with a justified recommendation.

TASK 7 – Evaluate Process Control System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how a command shell on the database server starts and manages programs.

Discuss:

- What fork(), exec(), wait() and exit() each do
- How a shell runs a command, and how it obtains the command's exit status
- What a zombie and an orphan process are, and how the OS deals with them
- Why fork() followed by exec() is used instead of a single "create process" call

Complete the Code

Complete TODO 1 below using fork(), execlp() and waitpid(), and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("shell: running job 1\n");
    fflush(stdout);                          /* empty the buffer before fork() */
    // TODO 1: Create a child with fork(). In the child, replace the program with
    //   execlp("expr", "expr", "6", "*", "7", (char *)NULL) and exit(127) if exec
    //   fails. In the parent, wait for the child with waitpid() and print "job 1 exit
    //   status N" using WEXITSTATUS



    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3);                  /* a child that fails with code 3 */
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Expected output:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Explain why stdout is flushed before each fork(). Then remove the second fflush(stdout), change _exit(3) to exit(3), redirect the output to a file and explain why "shell: running job 2" now appears twice.

Alternative Concept and Complexity Analysis

If commands were run with posix_spawn() or with threads instead of fork() followed by exec(), what would the solution be? Evaluate the time to start 1,000 commands, the memory copied or shared (copy-on-write), and the isolation between commands.

Evidence Expected

A working program that runs commands like a shell, with process diagrams and a reasoned evaluation.

TASK 8 – Evaluate CPU States and Protection — 3 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the hardware protects the hospital system.

Discuss:

- User mode and kernel mode, privileged instructions and the mode bit
- Interrupts, traps and the timer interrupt
- What would happen to the monitors if any program could disable interrupts
- How the hardware stops one program from reading another program's memory

Alternative Concept and Complexity Analysis

If the monitors used polling (checking the sensor in a loop) instead of interrupts, what would the solution be? Evaluate the worst-case delay before an abnormal reading is noticed (the polling interval versus the interrupt latency) and the CPU time and power wasted by polling.

Evidence Expected

A reasoned explanation of CPU modes, interrupts and protection, applied to the monitors.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how data moves between storage levels and devices.

Discuss:

- The levels of the memory hierarchy (registers, cache, main memory, SSD, hard disk, network storage) with typical access times and sizes
- Where MRI images, patient records and monitor readings should be kept
- Programmed I/O, interrupt-driven I/O and DMA, and which suits the MRI scanner
- The effect of caching on the speed of record look-up

Alternative Concept and Complexity Analysis

If the MRI scanner transferred images with interrupt-driven I/O (one interrupt per word) instead of DMA, what would the solution be? Evaluate the number of interrupts and the CPU time for a 500 MB image, and the memory buffers needed in each method.

Evidence Expected

A justified data-placement plan and I/O method for each device, using typical access times.

TASK 10 – Measure and Compare **4 Marks**

Bloom's Level: K5 – Evaluate

Measure the behaviour of your programs on a Linux system.

Discuss:

- The number of system calls made by your copy program for different buffer sizes (strace -c)
- The time spent in user mode and in kernel mode (the time command)
- The time to create 1,000 processes with fork() and to create 1,000 threads
- How the measurements support or contradict your earlier judgements

Alternative Concept and Complexity Analysis

If threads were created instead of processes for the measurements, what would the result be? Evaluate the creation time for 1,000 threads versus 1,000 processes and the memory used by each, and explain the difference using what each one shares.

Evidence Expected

Measured results in tables, compared with expectations, with explained differences.

TASK 11 – Recommend and Defend the Operating-System Plan **4 Marks**

Bloom's Level: K5 – Evaluate

Recommend the complete operating-system plan for the hospital.

Discuss:

- The type and structure of operating system for each environment
- The services, protection and I/O methods each will use
- The limitations and risks of your plan
- Whether the plan remains suitable for the group of three hospitals

Your recommendation must be based on **evidence from your own analysis and measurements**.

Alternative Concept and Complexity Analysis

If the whole system were moved to virtual machines on two large servers, instead of separate physical machines for each environment, what would the solution be? Evaluate the effect on response time for the monitors, the memory and processor overhead of virtualisation, and the effect on reliability.

Evidence Expected

A technically justified plan supported by analysis and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital adds **remote monitoring devices in patients' homes** that send readings over the internet, and moves patient records to a **cloud service** shared with the partner hospitals.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Identify which operating-system types, services and protection mechanisms are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your plan can support home devices and cloud records, considering response time, reliability and security.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe the revised plan (for example: an embedded real-time OS on home devices, a distributed system for shared records).

Evidence Expected

An appropriate design response.

D. Evidence-Based Defence – 2 Marks

Defend your decision using evidence and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating evidence and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Requirements of Each Environment	K5 – Evaluate	4
Task 2 – Types of Operating Systems	K5 – Evaluate	3
Task 3 – Operating-System Structures	K5 – Evaluate	4
Task 4 – Operating-System Services	K5 – Evaluate	4
Task 5 – System Calls	K5 – Evaluate	4
Task 6 – Cost of System Calls	K5 – Evaluate	4
Task 7 – Process Control System Calls	K5 – Evaluate	4
Task 8 – CPU States and Protection	K5 – Evaluate	3

Task	Bloom's Level	Marks
Task 9 – Memory Hierarchy and I/O	K5 – Evaluate	4
Task 10 – Measurement and Comparison	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including requirement tables, diagrams, strace and time outputs, measurements and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

ASSIGNMENT 1 (CO1) — ANSWER KEY / SOLUTION NOTES

These notes answer every task of Assignment 1 (Smart Hospital OS case study). They are written as a model solution to guide the student's own report, code execution, GitHub submission and viva preparation. Each student must still execute every program on their own Linux machine, capture real screenshots and strace/time output, and write the README.md as instructed on the cover pages.

TASK 1 – Evaluate the Requirements of Each Computing Environment (4 Marks)

Ranked requirement table (1 = most critical):

Bedside monitor: 1) Hard real-time response (<10 ms alarm), 2) Reliability/availability, 3) Security (cannot be tampered with), 4) Low power is secondary (mains-powered), throughput is irrelevant (one patient, few sensors). Nurse tablet: 1) Battery life (must last a full shift), 2) Usability/response time for the UI, 3) Wireless reliability, 4) Security (patient data on a mobile, portable device) is high but below battery/usability because the tablet is only a viewer, not the system of record. Doctor workstation: 1) Throughput for multiple applications, 2) Security/access control, 3) Response time for interactive use, 4) Reliability (a reboot is tolerable, unlike the monitor). Database server: 1) Reliability/availability (must never stop), 2) Data integrity/security, 3) Throughput (many concurrent queries), 4) Response time is important but secondary to never losing or corrupting a record. Imaging (MRI) server: 1) Throughput (CPU-bound image reconstruction), 2) Turnaround time, 3) Reliability, 4) Security of images. Partner-hospital link: 1) Security (data crosses an external network), 2) Reliability, 3) Throughput, 4) Response time.

Conflicts: the monitor needs a low, bounded worst-case response time, which conflicts with running many users/processes on one general-purpose machine, because a general-purpose scheduler only optimises average-case throughput and fairness, not worst-case latency — a single CPU-bound process (even at low priority) can still delay a time-slice due to interrupt masking, paging or priority inversion. Database "never stop" conflicts with doctor workstations' need for frequent software updates/reboots: this is solved by clustering/fail-over so maintenance never drops the service. High security (encryption, auditing) on the tablet conflicts with the limited battery/CPU budget, since cryptographic and logging overhead consumes power.

Safety-critical requirements: the monitor's 10 ms alarm bound and the database's never-stop requirement are directly tied to patient safety; everything else (tablet battery, workstation throughput) affects efficiency/convenience, not life-safety.

History lessons applied: batch systems teach that queued, non-interactive work (MRI reconstruction jobs) is fine to run as throughput-oriented batch jobs; time-sharing systems teach that many simultaneous users (workstations, tablets) need fair, responsive multiprogramming; personal/desktop OS history teaches that a workstation prioritises a rich UI and application variety over raw determinism; and the evolution from mobile/embedded RTOS history (motivated by exactly this kind of hard deadline) teaches that the monitor needs a dedicated real-time kernel, not a general-purpose time-sharing scheduler.

Alternative Concept and Complexity Analysis — one general-purpose OS everywhere: Worst-case alarm response time becomes unbounded: a standard (non-real-time) scheduler like Linux CFS gives no hard guarantee — the delay is O(number of runnable higher/equal priority tasks) and can be made arbitrarily large by interrupt storms, page faults or a priority-inverted lock, so the 10 ms deadline cannot be certified, only observed statistically. A real-time kernel with a fixed-priority pre-emptive scheduler (or on Linux, the SCHED_FIFO/SCHED_DEADLINE classes with PREEMPT_RT) bounds the delay to a calculable worst case (O(1) context-switch plus bounded interrupt latency). Memory needed on each device rises sharply if every monitor runs a full desktop OS (hundreds of MB to a few GB of RAM and flash) versus a few hundred KB–few MB for a stripped real-time/embedded kernel. Administration effort is lower with one OS image to patch (good), but patching 500 monitors still risks an alarm outage during an update window on a life-critical device, so uniformity trades safety for convenience — the better engineering answer is a small number of OS types, each

matched to its criticality class.

TASK 2 – Judge the Type of Operating System for Each Environment (3 Marks)

Justified choice per environment (rejected alternatives in brackets): • Bedside monitors → hard real-time, embedded OS (FreeRTOS/QNX/Zephyr) [rejected: desktop OS — no deadline guarantee; mainframe — wrong scale]. • Nurses' tablets → handheld/mobile OS (Android/iOS-class) [rejected: desktop OS — too power-hungry and heavyweight for a touch handheld]. • Doctors' workstations → desktop/multiprogrammed time-sharing OS (Windows/Linux desktop) [rejected: real-time OS — unnecessary restriction on general applications]. • Central database server → clustered OS/high-availability Linux cluster (e.g., Pacemaker/Corosync + clustered DB) [rejected: single desktop machine — single point of failure]. • Imaging (MRI) server → multiprocessor server OS, SMP Linux [rejected: handheld/real-time OS — wrong workload, needs throughput not determinism]. • Partner-hospital link → distributed OS/middleware over a secure WAN (standard server OS + distributed file/record-sharing protocol, e.g., VPN + replicated database) [rejected: clustered OS alone — clustering is for local fail-over, not geographically separate independent hospitals].

Symmetric vs asymmetric multiprocessing for the imaging server: SMP (symmetric) is preferred — every processor runs the kernel and can be scheduled any image-processing thread, giving load balancing and simpler scaling as more cores/processors are added; AMP (asymmetric), where one master processor runs the OS and others run only application code, is simpler to implement but wastes capacity and does not scale load evenly, so it is rejected for a workload (MRI reconstruction) with variable, parallelisable jobs.

Clustered database with fail-over: two or more servers share (or replicate) storage; a cluster manager monitors heartbeats and automatically restarts the database service/VIP on a surviving node within seconds, giving high availability without needing one perfect, never-failing machine — this directly satisfies the "must never stop" requirement that a single server cannot guarantee.

Alternative Concept and Complexity Analysis — single large multiprocessor instead of a cluster: Availability: a single big machine is a single point of failure (any hardware fault takes the whole database down until repaired, possibly hours); a cluster with fail-over restores service in seconds because a healthy node takes over. Cost: one large multiprocessor server (enterprise-grade, many sockets) typically costs far more per unit of throughput than several commodity servers of the same aggregate capacity, due to high-end CPU/RAS-feature pricing. Throughput growth: on a single SMP machine throughput grows roughly O(log n) to sub-linearly with processors added past a point, because of memory-bus and lock contention (Amdahl's law); a cluster can grow throughput closer to linearly for read-scaling workloads by adding nodes, though write-consistency overhead (replication) adds its own O(n) or O(n log n) coordination cost. In terms of memory, both need memory proportional to the active working set, but a cluster's replicated copies use roughly k× the space for k replicas, trading space for availability.

TASK 3 – Evaluate Operating-System Structures (4 Marks)

Structures compared: • Simple/monolithic — the whole OS (scheduler, file system, drivers, network stack) runs in one kernel address space. Fast (a single system call can call any kernel function directly, no message passing), but a bug in any driver can crash the whole kernel — poor reliability, hard to update one piece in isolation. • Layered — kernel is built as a stack of layers, each using only the layer below. Easier to verify and reason about correctness, but inter-layer calls add overhead and rigid layering can be hard to fit to real hardware. • Microkernel — only the minimal mechanisms (IPC, basic scheduling, minimal memory management) run in kernel mode; file systems, drivers and other services run as user-mode servers that communicate by message passing. High reliability/fault isolation (a crashed driver server can be restarted without crashing the kernel) and small trusted computing base, at the cost of message-passing performance overhead and more complex system design. • Modular (loadable kernel modules, LKM) — a monolithic core with dynamically loadable modules (Linux's approach). Combines monolithic performance with the flexibility to

add/remove drivers without a reboot, but a faulty module still runs in kernel space and can crash the system — weaker isolation than a true microkernel. • Hybrid — a mostly-monolithic kernel that borrows microkernel ideas for a few subsystems (Windows NT kernel, XNU/macOS). Balances performance and some modularity/isolation.

Effect on the hospital's two extreme environments: • Bedside monitor → a microkernel (or a small separation-kernel RTOS such as QNX) suits it best: the patient safety requirement (10 ms, never let one faulty component bring the whole device down) outweighs the small latency cost of message passing, and a crashed non-critical service (e.g., a display driver) must not kill the alarm logic. • Database server → a modular monolithic kernel (Linux) suits it best: raw I/O/file-system/network throughput matters more here than isolating individual driver crashes, and Linux's maturity, driver ecosystem and clustering tools (Pacemaker, DRBD) are built for exactly this role.

Real systems: Linux is a modular monolithic kernel; Windows NT is hybrid; Android is a modified Linux kernel (monolithic/modular) under a managed-runtime (ART) application layer; FreeRTOS is a minimal, single-address-space real-time kernel (essentially no user/kernel protection boundary, to minimise latency); QNX is a true microkernel, structured for exactly the fault-isolation the monitor needs.

Alternative Concept and Complexity Analysis — monolithic kernel instead of microkernel on the monitor: Time cost of a service request drops (one trap/system call directly reaches the needed kernel function, O(1) with a small constant) versus a microkernel's message pass-send-reply through an IPC server (extra context switches, still O(1) but with a larger constant — typically several times slower per call). Kernel memory footprint grows because all drivers/services are linked into one image instead of being started only as needed. Crucially, the effect of a faulty driver is now catastrophic: in a monolithic kernel a buggy driver runs with full kernel privilege and a bug (e.g., a null pointer dereference) can crash the entire device, silencing the alarm — in a microkernel the same bug only kills an isolated server process that can be restarted, so the alarm path keeps running. For a life-critical 10 ms alarm, this reliability difference outweighs the raw speed advantage of a monolithic design.

TASK 4 – Evaluate the Services an Operating System Must Provide (4 Marks)

Service table (✓ = included, ✗ = removed) and essentials for the database:

Bedside monitor — keep: program execution, minimal I/O (sensor + alarm), error detection, resource allocation (timers/interrupts), protection (so one task cannot corrupt the alarm task). Remove: full user interface (only a small status display, not a GUI shell), general file system (use raw flash/ring buffer instead of a full POSIX FS), general networking stack (only a thin telemetry link), accounting/billing services, multi-user login. Removing these shrinks kernel/image size and removes unnecessary attack surface and scheduling noise, which both speeds up and shrinks the device.

Database server — essential: file system (durable record storage), protection and security (role-based access, encryption at rest/in transit), error detection (checksums, RAID, ECC), resource allocation (scheduling many concurrent queries fairly), communication (network stack for client connections and replication), accounting/auditing (who accessed which record, required for medical-record compliance). User-interface services are the least essential (headless server).

OS protection of one patient's records from another user's programs: the OS's memory-protection hardware (user/kernel mode bit, per-process virtual address spaces with MMU-enforced page tables) stops one process from directly addressing another's memory; the file system's permission/ACL layer and the database's own authentication and row/role-level access control stop one user's queries from reading another user's data even though both run as OS processes on the same machine; auditing and logging detect and record any attempted violation.

Alternative Concept and Complexity Analysis — tablet with a full desktop OS instead of a reduced handheld OS: Space — a full desktop OS and its services (full windowing system, many background daemons, full

printing/Bluetooth/legacy-driver stacks) use hundreds of MB to a few GB more RAM/storage than a purpose-built handheld OS that loads only the services the clinical app needs. Time — start-up time rises from a few seconds (handheld OS, optimised boot path) to tens of seconds to minutes (desktop OS with many services to initialise), which matters when a nurse needs the tablet instantly at the bedside. Battery life falls because of (a) extra background services polling/waking the CPU and (b) a desktop OS's power management is tuned for mains-powered or laptop use, not for an all-day, always-on clinical handheld — overall a materially shorter shift-length battery life, directly contradicting the staff's stated concern.

TASK 5 – Examine System Calls (4 Marks)

Categories of system calls: (1) Process control — fork, exec, exit, wait, kill; (2) File management — open, read, write, close, lseek; (3) Device management — ioctl, read/write on device files; (4) Information maintenance — getpid, time, sleep; (5) Communication — pipe, socket, send/recv, shmget; (6) Protection — chmod, chown, setuid.

Step-by-step for read(): (1) The application calls the C-library wrapper read(fd, buf, n) — an ordinary function call in user mode. (2) The wrapper loads the system-call number and arguments into registers (or a parameter block) and executes a trap instruction (e.g., syscall on x86-64). (3) The trap raises the CPU privilege level from user mode to kernel mode and transfers control to a fixed kernel entry point (the mode switch; the hardware also saves the user-mode program counter/registers). (4) The kernel's system-call dispatcher uses the system-call number to index a table of handler functions and calls the real read() kernel routine, which validates the file descriptor and buffer, copies data from the kernel's page cache/device buffer into the user buffer, and updates the file offset. (5) The kernel places the return value (bytes read, or -1 with errno) back in registers, restores the saved user-mode context, and executes a return-from-trap instruction, switching the CPU back to user mode and resuming the C library wrapper, which returns the value to the application.

Parameter passing: on Linux x86-64 the first six arguments are passed in fixed registers (rdi, rsi, rdx, r10, r8, r9) and the system-call number in rax; older/other ABIs may pass parameters in a block of memory pointed to by a register, or push them on the stack — the exact convention is architecture- and OS-specific, but the principle is always "argument values are placed somewhere the trap handler can find them deterministically before the trap instruction executes".

strace: running strace -c ./program (or strace ./program for a full trace) intercepts every system call the traced process makes, printing the call name, its arguments, its return value and (with -c) a summary table of call counts, total time and errors per call — this is exactly the tool Task 6/10 use to verify how many read()/write() calls a program actually issued.

Alternative Concept and Complexity Analysis — fopen/fread/fwrite instead of open/read/write directly: the C standard library adds a user-space buffer (commonly 4096 bytes, i.e. BUFSIZ) in front of the raw system calls, so for a file of n bytes the number of underlying read()/write() system calls falls from n (if the program naively called read() one byte at a time) to about ceil(n / 4096); more generally, for an explicit buffer of b bytes chosen by the programmer, the count is about ceil(n / b) — this matches exactly the measurement the TODO-1 program produces. The trade-off is extra buffer space (O(b) per open stream, held in user memory) and a memory-copy from the library's buffer into the caller's variables, in exchange for far fewer expensive kernel transitions — this is precisely why Task 6 shows the system-call count falling sharply as buffer size grows.

TASK 6 – Evaluate the Cost of System Calls (4 Marks) — Complete the Code

Why a system call costs more than an ordinary function call: an ordinary function call is just a branch/return at the same privilege level with no state saved beyond the stack frame (a few CPU cycles). A system call must trap into the kernel: save the full user register set, switch the privilege level and page table/TLB context in some cases, execute the kernel handler, validate all user-supplied pointers for safety, then restore the user context and switch back — this is typically on the order of 100–1000+ CPU cycles, even before the kernel does any

real work, which is one to two orders of magnitude more expensive than a plain function call.

Buffer size vs. number of calls: with a buffer of b bytes, copying a file of n bytes takes ceil(n / b) read() calls and ceil(n / b) write() calls, so halving b roughly doubles the number of system calls (and vice-versa) — this is the direct mechanism Task 6's program measures (1-byte buffer → 10001 calls, 512-byte buffer → 21 calls, 4096-byte buffer → 4 calls for a 10,000-byte file).

Library buffering: fread/fwrite batch many small application-level reads/writes into one internal BUFSIZ-sized buffer and only issue the underlying read()/write() system call when that buffer is full/empty, so an application that calls fgetc() in a loop still only makes about n/BUFSIZ real system calls, not n.

Best buffer size for the backup program: in practice, throughput rises steeply as the buffer grows from a few bytes up to roughly the storage device's/page-cache's natural transfer unit (commonly in the 4 KB–256 KB range), then levels off — beyond that point the fixed per-call overhead is already amortised and the transfer is dominated by actual I/O bandwidth, so there is little further gain and only extra memory cost; a good practical default (and the one measured in Task 6/10) is a buffer in the 64 KB–1 MB range, to be confirmed against the student's own measured numbers.

```
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }

    long calls = 0;
    ssize_t n;

    /* TODO 1 solution */
    while ((n = read(in, buf, bufsize)) > 0) {
        calls++;
        ssize_t written = 0;
        while (written < n) {
            ssize_t w = write(out, buf + written, n - written);
            if (w < 0) { close(in); close(out); return -1; }
            written += w;
        }
    }
    if (n < 0) { close(in); close(out); return -1; }

    close(in);
    close(out);
    return calls;
}
```

Verification: compiling with gcc -std=c11 -Wall -Wextra -o copytest copytest.c and running it reproduces the expected output (buffer 1 byte → 10001 read() calls, 512 bytes → 21 calls, 4096 bytes → 4 calls). Running strace -c ./copytest shows the count of read/write calls in the "calls" column, which should match the program's own printed counts exactly (strace counts every real system call, so it is an independent confirmation that the TODO was implemented correctly and that no extra/short reads are occurring).

Alternative Concept and Complexity Analysis — mmap() instead of read()/write() with a buffer: mmap() maps the whole source file (or large chunks of it) directly into the process's virtual address space; copying then becomes a small, fixed number of system calls (one or two mmap/munmap/msync calls regardless of file size) instead of about n/b read+write calls — a reduction from O(n/b) to O(1) system calls. It also avoids one of the two kernel-to-user memory copies that read()/write() perform (the mapped pages are accessed directly, backed by the page cache, so for read-only access there is effectively zero extra copy, only a page fault the first time each page is touched). For very large files, mmap uses only as much resident memory as the pages actually touched (demand-paged), but the full file must fit in the process's virtual address space, and on 32-bit systems a file much larger than ~2–3 GB cannot be mapped in one go, whereas buffered read/write has no such limit since it only ever holds one buffer's worth of data in memory at a time.

TASK 7 – Evaluate Process Control System Calls (4 Marks) — Complete the Code

fork(): creates a new process (the child) that is an almost-exact copy of the calling (parent) process's address space, open file descriptors and execution point; it returns twice — 0 in the child and the child's PID in the parent (or -1 on failure). exec() family: replaces the calling process's program image (code, data, stack) with a new program, while keeping the same PID and most open file descriptors; it does not return on success because the old program no longer exists in that process. wait()/waitpid(): suspends the calling (parent) process until one of its children changes state (terminates, by default), and retrieves the child's exit status so the OS can release the child's remaining kernel bookkeeping (the PCB/zombie entry). exit(): terminates the calling process, flushes stdio buffers (exit(), unlike _exit(), runs atexit handlers and flushes C library buffers), closes its file descriptors, and leaves an exit status for the parent to collect via wait().

How a shell runs a command: it calls fork() to create a child, the child calls one of the exec() functions to load the requested program image, and the parent calls waitpid() to block until the child finishes and then reads the exit status with the WEXITSTATUS() macro to report/act on it (e.g., $? in a shell).

Zombie and orphan: a zombie is a process that has called exit() but whose parent has not yet called wait()/waitpid() — the kernel keeps a minimal PCB entry (PID, exit status) until the parent reaps it; an orphan is a process whose parent has terminated before it did — the kernel re-parents orphans to init (PID 1) or a subreaper, which calls wait() on their behalf so they do not become permanent zombies.

Why fork()+exec() instead of one "create process" call: splitting process creation into fork() (duplicate) and exec() (replace the image) lets the child adjust its own environment — redirect stdin/stdout/stderr, change its working directory, close or duplicate file descriptors, drop privileges, set resource limits — all using ordinary system calls executed between the fork() and the exec(), before the new program starts; a single combined call would need a huge, inflexible parameter list to cover every such case, whereas fork()+exec() composes cleanly with everything else the OS already provides.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("shell: running job 1\n");
    fflush(stdout); /* empty the buffer before fork() */

    /* TODO 1 solution */
    pid_t pid1 = fork();
    if (pid1 == 0) {
        execlp("expr", "expr", "6", "*", "7", (char *)NULL);
        exit(127); /* only reached if exec fails */
    }
    int status1;
    waitpid(pid1, &status1, 0);
    printf("job 1 exit status %d\n", WEXITSTATUS(status1));

    printf("shell: running job 2\n");
    fflush(stdout);

    pid_t pid;
    int status;
    pid = fork();
    if (pid == 0) _exit(3); /* a child that fails with code 3 */
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Why stdout is flushed before each fork(): stdio applies user-space buffering, so text that has been printed but not yet flushed still sits only in the parent's C-library buffer, not in the kernel/the terminal. fork() duplicates that buffer into the child's address space exactly as it stood; if the buffer is not flushed first, both parent and child

inherit the same pending (unflushed) bytes, and whichever of them later flushes (or exits via exit(), which flushes automatically) will print that pending text again, duplicating output. Flushing before fork() empties the buffer so nothing is left to be duplicated.

Removing the second fflush(stdout), changing _exit(3) to exit(3), and redirecting output to a file: when stdout is redirected to a regular file (not a terminal), stdio switches from line-buffered to fully block-buffered, so "shell: running job 2\n" is written into the buffer but not flushed to the file immediately. fork() then duplicates that still-unflushed buffer into the child. The child, instead of calling _exit(3) (which bypasses stdio clean-up), now calls exit(3), which flushes its inherited copy of the buffer — printing "shell: running job 2" from the child. The parent later also exits normally (returning from main), flushing its own copy of the same unflushed line — printing it a second time. The result is that "shell: running job 2" appears twice in the output file, once from each process's independent copy of the buffer.

Alternative Concept and Complexity Analysis — posix_spawn() or threads instead of fork()+exec(): posix_spawn() creates and execs a new process in one call (internally often implemented with vfork()-like semantics or clone() with tighter flags), avoiding the cost of duplicating a large parent address space that fork() would otherwise copy-on-write; for starting 1,000 short-lived commands this is typically noticeably faster in wall-clock time than fork()+exec(), because fork() still has some fixed per-call overhead (duplicating page tables, file-descriptor tables) even though copy-on-write defers copying page contents themselves. Using threads instead of separate processes to "run commands" is not equivalent for isolation — a thread cannot safely exec() a replacement program for just itself without destroying the whole process (exec() replaces the entire address space, all threads included), so true command isolation requires processes; threads only make sense for the alternative in Q1/Q3 of Assignment 2, where the work is in-process computation rather than running separate external programs. Memory: fork() shares pages copy-on-write (so the initial cost is proportional to page-table size, O(address-space pages), not full data copy); posix_spawn() (POSIX_SPAWN_USEVFORK-style) can avoid even that COW setup since the child execs almost immediately. Isolation: fork()+exec() and posix_spawn() both give full process isolation (separate address space after exec); threads share one address space and offer no isolation between "commands".

TASK 8 – Evaluate CPU States and Protection (3 Marks)

User mode and kernel mode: the CPU has a hardware mode bit; in user mode, privileged instructions (changing the page table base, masking interrupts, accessing I/O ports directly, halting the CPU) are forbidden and trap to the kernel; in kernel mode all instructions are permitted. This two-state (or multi-ring) design means user programs can run application code freely but can never directly damage the OS or other processes' resources; they must ask the kernel to do privileged work for them via a system call.

Interrupts, traps and the timer interrupt: an interrupt is an asynchronous signal from hardware (a device finishing an I/O, or a periodic timer) that pre-empts the CPU to run a kernel interrupt handler; a trap (or exception) is a synchronous event caused by the currently executing instruction itself (a system call, a divide-by-zero, a page fault). The timer interrupt fires at a fixed, periodic interval and is what lets the kernel regain control of the CPU even from a program that never calls a system call or yields — the timer handler can decide to pre-empt the running process and run the scheduler, which is what makes pre-emptive multitasking possible at all.

If any program could disable interrupts on the monitors: a buggy or malicious user program could mask the timer interrupt (and any sensor-data interrupt) and simply never return control to the kernel, starving the alarm logic indefinitely — the very 10 ms deadline the hospital depends on would become unenforceable, because the hardware mechanism that guarantees the kernel eventually regains the CPU would be gone. This is exactly why disabling interrupts is a privileged instruction, permitted only in kernel mode and normally only for very short, bounded critical sections inside the kernel itself.

How hardware stops one program reading another's memory: each process is given its own virtual address space; the CPU's memory-management unit (MMU) translates every virtual address a running program issues

through that process's current page table, and any address that is not mapped in that table (including another process's physical pages) causes a page fault/protection fault that traps to the kernel rather than succeeding — a program literally has no instruction that can name another process's physical memory address unless the OS has explicitly mapped a shared page into both page tables.

Alternative Concept and Complexity Analysis — polling instead of interrupts on the monitors: with polling, the worst-case delay before an abnormal reading is noticed equals the polling interval (if the sensor loop checks every p ms, the worst case is up to p ms of extra delay on top of any processing time) — and if p is pushed below 10 ms to make the deadline, the CPU spends almost 100% of its time in the polling loop, at the expense of every other task on the device, whereas interrupt-driven I/O only consumes CPU time when an event actually occurs (near 0% idle overhead) and can still achieve microsecond-scale hardware interrupt latency, far tighter than a coarse polling interval. In terms of power, continuous polling keeps the CPU in an active (non-idle) power state constantly, wasting significant energy on a battery- or low-power device, while interrupt-driven I/O lets the CPU sleep in a low-power idle state between events — so for the monitor's hard deadline and (where battery-powered, e.g. portable monitors) power budget, interrupts strictly dominate polling.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems (4 Marks)

Memory hierarchy (typical values, illustrative order of magnitude): Registers: <1 ns access, tens of bytes. CPU cache (L1/L2/L3): ~1–20 ns, tens of KB to tens of MB. Main memory (DRAM): ~50–100 ns, GBs. SSD: ~10–100 µs, hundreds of GB–TBs. Hard disk: ~5–10 ms (dominated by seek + rotation), TBs. Network storage/cloud: ~1–100+ ms depending on link, effectively unlimited capacity. Each level down is roughly an order of magnitude slower and an order of magnitude larger/cheaper per byte than the level above.

Data placement: monitor readings (small, latency-critical, short retention) → kept in fast local RAM/a small ring buffer, persisted to local flash only periodically; patient records (must never be lost, accessed unpredictably by many staff, long retention) → on the reliable, redundant database server's SSD-backed storage (with RAID and backups), cached in RAM (page cache/DB buffer pool) for frequently accessed charts; MRI images (very large, written once, read occasionally, not latency-critical per read) → bulk storage (hard disk array or object/cloud storage), with SSD or RAM caching only for currently-open studies.

I/O methods: programmed I/O (CPU polls a status register and transfers every word itself) wastes CPU and is unsuitable for bulk transfer; interrupt-driven I/O (device interrupts the CPU for each unit transferred) is better than polling but still burdens the CPU with per-word/per-small-block interrupts; DMA (Direct Memory Access — a DMA controller moves a whole block directly between device and memory, interrupting the CPU only once per block, or once per transfer) is the right choice for the MRI scanner, because MRI images are large (hundreds of MB) and transferring them word-by-word through the CPU would consume enormous CPU time for no benefit — DMA frees the CPU to do other work during the transfer and only interrupts once completion.

Caching's effect on record look-up: once a record (or index page) is cached in RAM, subsequent look-ups avoid the disk/SSD access time entirely, cutting look-up latency by roughly the ratio of disk/SSD access time to RAM access time (often 1,000× to 100,000× faster for a cache hit versus a cold access), which is why database buffer pools and OS page caches are central to interactive record-retrieval performance.

Alternative Concept and Complexity Analysis — interrupt-driven I/O (one interrupt per word) instead of DMA for the MRI scanner: for a 500 MB image, interrupt-driven I/O at (say) 4 bytes/word would require roughly $500 \times 1024 \times 1024 / 4 \approx 131$ million interrupts, each costing a context switch/handler overhead typically in the hundreds of ns to low-µs range — tens to over a hundred seconds of pure CPU interrupt overhead for one image, compared to DMA's single (or a handful of) interrupt(s) for the whole transfer, whose CPU overhead is negligible by comparison (O(1) interrupts vs O(image size / word size)). Memory buffers: both methods need a buffer the size of one transfer unit per step, but interrupt-driven I/O's CPU is fully occupied servicing interrupts throughout the transfer, whereas DMA needs only a contiguous (or scatter-gather) memory region set up once and is otherwise CPU-free during the transfer — so DMA is essential, not optional, for a device moving data at this scale.

TASK 10 – Measure and Compare (4 Marks)

This task is primarily experimental; the table below shows the pattern of results a correct implementation should produce (exact numbers depend on the student's own CPU, Linux distribution and load, and must be captured from the student's own machine for the submission).

System-call counts by buffer size (from Task 6's copy_file, confirmed by strace -c): the count of read()/write() pairs falls roughly as n/b as the buffer size b grows (e.g., 1 byte ≈ 10,001 calls, 512 bytes ≈ 21 calls, 4096 bytes ≈ 4 calls, for a 10,000-byte file) — strace -c's own "calls" column for read should equal the value the program itself reports.

User vs kernel time (the time command): expect "real", "user" and "sys" times to be reported; for a small, CPU-light I/O-bound copy, "sys" time (kernel-mode CPU, i.e. time spent inside system calls) should dominate relative to "user" time when the buffer is tiny (many system calls, little real computation per call), and "user" time should shrink to near zero as the buffer grows and the number of system calls falls — demonstrating directly that the per-call kernel-entry overhead, not the data movement itself, is the dominant cost for small buffers.

Creating 1,000 processes (fork()) vs 1,000 threads (pthread_create()): expect thread creation to be measurably faster per-unit (often several times faster) than process creation, because fork() must set up a new address space (new page tables, duplicate file-descriptor table, copy-on-write bookkeeping) while pthread_create() only allocates a new stack and kernel scheduling entity within the same, already-existing address space — and expect thread creation to use far less memory per unit (just a stack, often a few hundred KB to a few MB depending on default stack size) versus a full process (page tables + its own, albeit mostly copy-on-write-shared, address-space bookkeeping).

How measurements support/contradict earlier judgements: measured results should support Task 6's prediction that larger buffers sharply reduce system-call count and kernel time, and should support Q1/Q3 of Assignment 2's expectation that threads are cheaper to create than processes — if a student's own numbers disagree with the predicted trend, the report must explain why (e.g., system load, caching effects, measurement noise) rather than silently ignoring the discrepancy.

Alternative Concept and Complexity Analysis — threads instead of processes for the measurement: creating 1,000 threads is expected to be faster than creating 1,000 processes because a thread only needs a new stack and a lightweight kernel scheduling entity inside an already-mapped address space (no new page table, no duplicated file-descriptor table, no copy-on-write set-up), whereas fork() must duplicate several per-process kernel structures even under copy-on-write. Memory used: a process needs its own page tables plus (until written) copy-on-write-shared data pages, which is more kernel bookkeeping overhead than a thread's stack-only cost; 1,000 threads in one process will typically use noticeably less total memory than 1,000 separate processes. The difference comes down to what each one shares: threads share the same code, data, heap and file descriptors (only the stack, registers and a few per-thread kernel structures are private), while processes share nothing by default (each has its own private address space), so process creation inherently duplicates more state than thread creation.

TASK 11 – Recommend and Defend the Operating-System Plan (4 Marks)

Final recommended plan (summary, consistent with Tasks 1–10): • Bedside monitors — hard real-time embedded OS (e.g., FreeRTOS/QNX-class), microkernel/separation-kernel structure, minimal service set (no general file system/UI/network stack beyond a thin telemetry link), interrupt-driven alarm path, DMA not required at this scale, mandatory authentication on any maintenance interface. • Nurses' tablets — mobile/handheld OS, reduced service set (only the clinical app's needed services enabled), Wi-Fi link to the hospital network with encryption, battery-aware scheduling. • Doctors' workstations — general-purpose desktop OS, full service set, strong access-control/auditing integrated with hospital identity management. •

Database server — clustered, modular-monolithic (Linux) OS with fail-over clustering (Pacemaker/DRBD-style), RAID-protected storage with page-cache/buffer-pool caching, encryption at rest and in transit, comprehensive auditing. • Imaging server — SMP multiprocessor server OS (Linux), DMA-based I/O for scanner transfers, bulk storage for images with caching for open studies. • Partner-hospital link — secure distributed middleware (encrypted VPN/replicated database) between independently clustered hospital systems, not a single shared cluster.

Limitations and risks: heterogeneity increases the number of OS images to patch/certify; the real-time monitor OS is a specialised skill-set to maintain; cluster fail-over must itself be tested regularly or the "never stop" guarantee is only theoretical; the partner link is the largest attack surface and single external dependency.

Suitability for a group of 3 hospitals: the plan scales horizontally — more monitors/tablets/workstations of the same type can simply be added; the database cluster and imaging SMP servers can be scaled up (more nodes/processors) or out (more clusters, one per hospital, replicating between them); the main new requirement at 3-hospital scale is a true distributed/replicated record-sharing design (not just a VPN link between two clusters), reinforcing that the partner link should already have been built as a distributed system rather than a point-to-point connection, so it remains suitable with a config change rather than a redesign.

Alternative Concept and Complexity Analysis — virtual machines on two large servers instead of separate physical machines per environment: for the monitors specifically, virtualising a hard real-time workload adds the hypervisor's own scheduling layer on top of the guest OS's scheduler, which typically increases and, worse, makes less predictable the worst-case interrupt/response latency (the "noisy neighbour" effect from other VMs sharing the same physical CPU) — this directly threatens the 10 ms deadline, so real-time monitor workloads are generally kept off shared general virtualisation unless a real-time hypervisor with CPU pinning/partitioning is used. Memory/processor overhead: virtualisation adds a few percent to double-digit percent CPU overhead (hardware-assisted virtualisation such as Intel VT-x/AMD-V keeps this fairly low for compute-bound workloads like imaging) plus the memory cost of each guest's own kernel and services, versus zero such overhead on bare metal. Reliability: consolidating many roles onto "two large servers" reintroduces a smaller number of larger single points of failure — losing one physical host now risks several services at once unless VM live-migration/HA is itself clustered across the two hosts, which is exactly the fail-over problem Task 2 already solved for the database alone and would now need to be solved hypervisor-wide.

TASK 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis (2 Marks) The changed requirement (home remote-monitoring devices over the public internet, and cloud-hosted shared records) affects: (i) OS type — home devices need a small, low-power embedded/IoT OS (similar family to the bedside monitor but now internet-facing, not an isolated hospital LAN device); (ii) services — a real network/communication stack and TLS/security services must now be added to a device class that previously only needed a thin, trusted local telemetry link; (iii) protection mechanisms — the trust boundary moves: previously the hospital network perimeter protected the monitor-to-database path, but home devices now traverse the untrusted public internet, so authentication, encryption and intrusion detection become mandatory at the device, not just at the hospital gateway; (iv) the database/record system changes from a single clustered on-premises system to a distributed system that must synchronise with a cloud service and with partner hospitals.

B. Evaluation of the Original Design (2 Marks) Response time: the original plan assumed a 10 ms bound only within the hospital's own trusted LAN; a home device's internet path has far higher and more variable latency, so the original design's real-time guarantee cannot be extended to home monitoring as-is — home monitoring must be re-classified as "soft real-time, best-effort with local safety fallback" rather than inheriting the bedside monitor's hard 10 ms bound. Reliability: the original clustered database assumed hospital-controlled infrastructure; a cloud records service introduces a new external dependency (the cloud provider's own availability) that the original plan never accounted for. Security: the original design's security model (network perimeter + internal access control) is insufficient once records are shared with a third-party cloud and partner hospitals — it needs end-to-end encryption, stronger authentication (the home device cannot be physically

secured like a bedside unit) and a clear data-sharing/trust agreement with the cloud provider and partner hospitals.

C. Adaptation or Redesign (2 Marks) — MODIFY, not full redesign. • Home devices: a small embedded real-time OS (same family as the bedside monitor) extended with a secure, authenticated, encrypted communication stack (e.g., TLS over a cellular/Wi-Fi link) and a local safety fallback (the device itself raises a local alarm — audible/visual/caregiver SMS — if connectivity is lost, rather than relying solely on round-trip confirmation from the hospital). • Records: extend the existing clustered database into a distributed, replicated system that also synchronises with the cloud records service and partner hospitals, using eventual consistency for cross-site replication (acceptable, since most record reads do not require millisecond cross-site consistency) while keeping the existing strongly-consistent cluster for the hospital's own primary, latency-sensitive operations. • Add a secure API gateway/edge layer between home devices, the cloud service and the hospital network, so the internal hospital systems are never directly exposed to the public internet.

D. Evidence-Based Defence (2 Marks) This is a MODIFY rather than a full REDESIGN because the core type/structure choices from Tasks 1–11 remain correct for every existing (in-hospital) component — only the home-device and records-sharing edges need new capabilities layered on top. The trade-off accepted is slightly higher and more variable response time and a larger security attack surface for home monitoring and cloud records, in exchange for the clinical benefit of remote monitoring and shared records across hospitals; this is mitigated by keeping the hard real-time guarantee local to the hospital (where it can actually be bounded) and treating home monitoring as soft real-time with a local safety fallback, and by adding encryption/authentication precisely at the new trust boundaries identified in part A, rather than weakening any of the original, still-valid in-hospital guarantees.